

03

IDEMPOTENCY KEYS

Two requests failed.
The customer paid twice.

WHAT THE CLIENT SAW

```
$ curl --max-time 10 -X POST /v1/charges  
  
curl: (28) Operation timed out after 10002 ms  
with 0 bytes received  
  
http_status=000  
exit code: 28
```

Nothing came back. No transaction id, no status, no error body.
So the client did the only reasonable thing. It tried again.

THIS LOOKS FINE

```
public ChargeResponse charge(String apiKey, ChargeRequest req) {
    UUID merchantId = db.queryForObject("""
        SELECT merchant_id FROM api_keys WHERE key_hash = ?
        """, UUID.class, ApiKeyHasher.hash(apiKey));

    UUID txId = db.queryForObject("""
        INSERT INTO transactions (merchant_id, reference,
            amount, currency, status, provider)
        VALUES (?, ?, ?, ?, 'PENDING', 'stub') RETURNING id
        """, UUID.class, merchantId, req.merchantReference(),
            req.amount(), req.currency());

    ProviderResult result = provider.charge(req);

    if (result.status().equals("SUCCESS")) {
        creditMerchant(merchantId, txId, req);
    }
    db.update("UPDATE transactions SET status = ? WHERE id = ?",
        result.status(), txId);
    return new ChargeResponse(txId, req, result.status());
}
```

Validate. Insert. Call the provider. Write the ledger. Return. There is no wrong line in here.

HERE'S WHAT HAPPENS

reference	amount	status	created_at
-----+-----+-----+-----			
dup-charge-demo	4500000	SUCCESS	15:16:41.986335
dup-charge-demo	4500000	SUCCESS	15:16:55.969022
(2 rows)			
ledger_entries	direction	amount	
-----+-----+-----			
85435e3d...	CREDIT	4500000	
85435e3d...	DEBIT	4500000	
95cda51a...	CREDIT	4500000	
95cda51a...	DEBIT	4500000	
(4 rows)			
accounts.balance	before: 0		
	after: 9000000		

₦45,000, charged twice, fourteen seconds apart. Both requests returned nothing.

EVERY INTEGRITY CHECK PASSED

CHECK	RESULT
ledger_entries = 2 × SUCCESS	PASS
exactly 1 DEBIT + 1 CREDIT	PASS
sum(DEBIT) = sum(CREDIT)	PASS
no rows for FAILED / PENDING	PASS

The books balanced. The pairs were correct. Nothing was orphaned.

A consistent ledger cannot tell you a charge should never have existed. Integrity checks verify arithmetic, not intent.

THE FIX

```
String fingerprint = idempotency.fingerprint(merchantId, path, req);

if (idempotency.claim(merchantId, key, fingerprint)) {
    ChargeResponse charged = chargeService.charge(apiKey, req);
    idempotency.complete(merchantId, key, 201, body);
    return json(201, body);
}

IdempotencyRecord prior = idempotency.find(merchantId, key);

if (!prior.fingerprint().equals(fingerprint))
    return problem(422, "key reused with a different payload");

if (prior.status().equals("IN_FLIGHT"))
    return status(409).header("Retry-After", "5");

return replay(prior);    // stored status + stored body
```

claim() is `INSERT ... ON CONFLICT DO NOTHING`, committed on its own **before** the provider call.

Claim afterwards and you have protected nothing — the duplicate arrives inside the window you left open.

SAME SCRIPT. SAME 30S PROVIDER.

```
= request 1 =  
curl: (28) timed out after 10006 ms, 0 bytes received  
  
= request 2 (byte-identical) =  
{"error": "Original request is still in flight"}  
http_status=409    time_total=0.051032s
```

	BEFORE	AFTER
transactions	2	1
ledger_entries	4	2
balance	9,000,000	4,500,000

The duplicate was rejected in 51ms, while the first charge still had 16 seconds to run.

THE TRADE-OFF

GAINED

- No duplicate charge, even mid-flight
- Retries become safe by default
- Replay returns the original response

COST

- A table on the hot path
- You must define "the same request"
- A crash leaves a key IN_FLIGHT until TTL

And the client still received nothing. Idempotency did not fix their uncertainty — it fixed the merchant's money.

Post 04, Friday — the double-entry ledger, and why the balanced pairs above are a lie.